

P2L1 Review of UML

Diagrams provide an important and useful method for illustrating design structure beyond written descriptions.

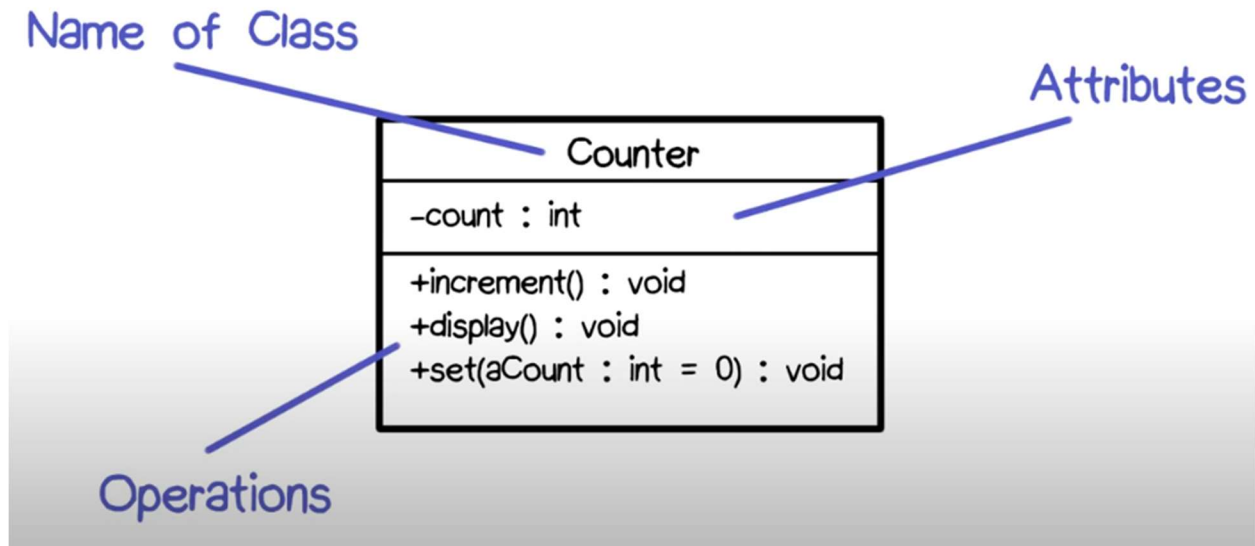
Object Management Technique (OMT) – developed by James Rumbaugh and others at GE, showing three sides/faces of a cube with different “views” of a system – a class model diagram, state chart diagram, and data flow diagram. The class model diagram gives the structural aspects of the design, the state chart diagram gives the behavioral aspects of the design, and the data flow diagram gives the functional aspects of the design.

Unified Modeling Language (UML) – created by Rumbaugh, Booch, and Jacobson, who each published a book (user manual, guidebook, etc.) separately that describe UML completely when taken together. UML is standardized by the Object Management Group (OMG) and pulls together different styles and techniques from many design schemas that came out around the same time (including OMT). There are many different tools for creating these diagrams, including some free tools, and the diagrams can be used for both design and analysis of models – design looking forward to a solution for a system/problem, analysis looking backward to a concrete, implemented system that can be depicted as a diagram after-the-fact to help with documentation, maintenance, etc.

UML encompasses fourteen (14) different types of diagrams, but most will not be used for any particular job or project – only the trickiest/most complex part of a system to be better planned/explained. Certain diagrams tend to be used more commonly, and the main categories of UML diagrams are “structural” and “behavioral.” Structural diagrams give you the pieces of the system that are always there and the relationships among them. Behavioral diagrams are concerned with the executions of the system, and a particular diagram may only convey one execution – for example, you may need multiple sequence diagrams to give complete coverage of all the behaviors of a system across multiple different executions.

Class model diagram – most popular UML diagram, shows the structure of a system through classes and their relationships.

UML Classes



UML classes are the main component of class diagrams, along with relationships between these classes.

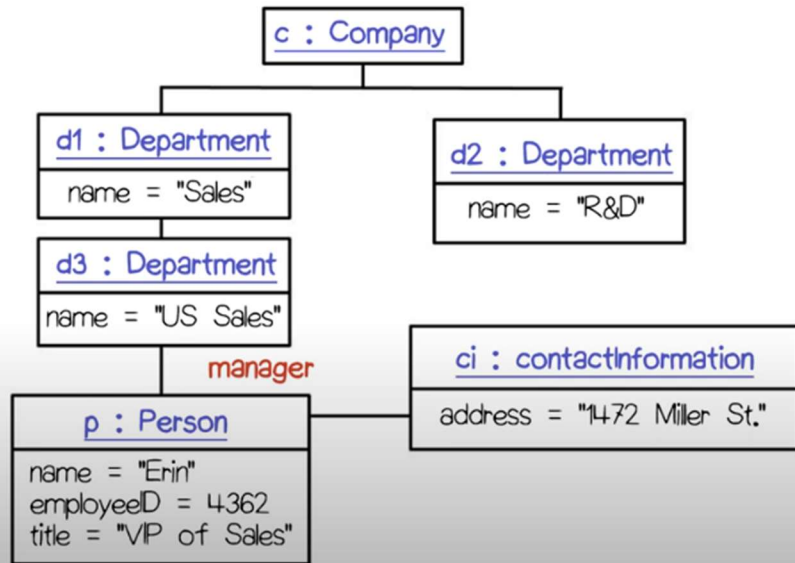
Relationships

- **Dependency:** X uses Y 
Dashed, directed line
- **Associations:** X affects Y 
Solid undirected line
- **Generalization:** X is a kind of Y 
Solid line with large, open arrowhead

Relationships between classes in class diagrams tend to fit into one of the above categories, and are depicted on the class diagram using the associated line marking.

Object Diagram

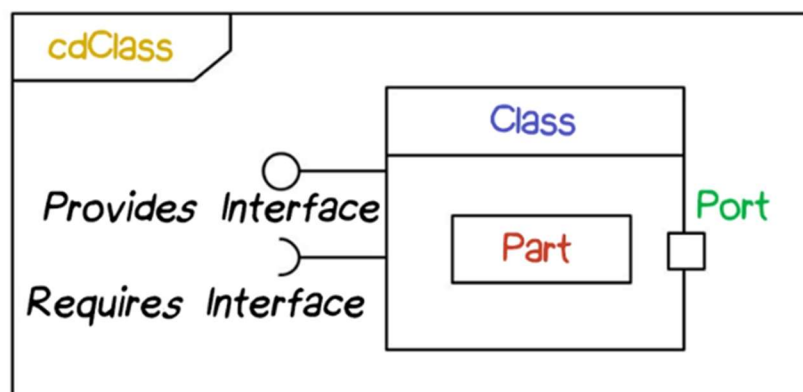
Conveys **objects** and **links** instead of **classes** and **relationships**



Object diagram – almost identical to the class diagram, but with actual instances of objects and their attribute values filled in to represent the state of a system (from an object standpoint)

Composite Structure Diagram

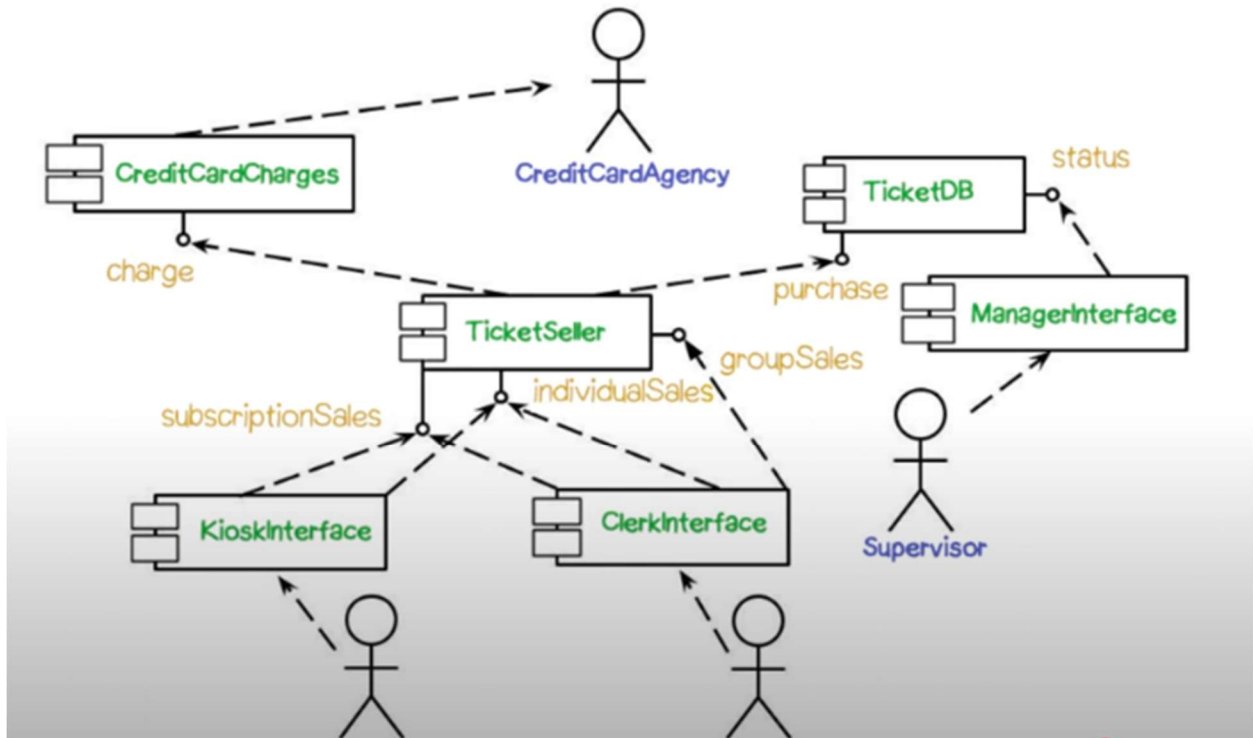
Conveys **internal structure** of a class



Composite Structure diagrams show the inner implementation and structure of a class, and how it interrelates to other classes (using “interfaces” – like APIs). These are less common than the other class diagrams but show a more detailed view of the structure of the system.

Component Diagram – a static implementation view of the system. According to the UML Reference Manual, “A physical, replaceable part of a system that packages implementation and conforms to and provides the realization of a set of interfaces.” The component diagram models code entities, and is used to convey architecture.

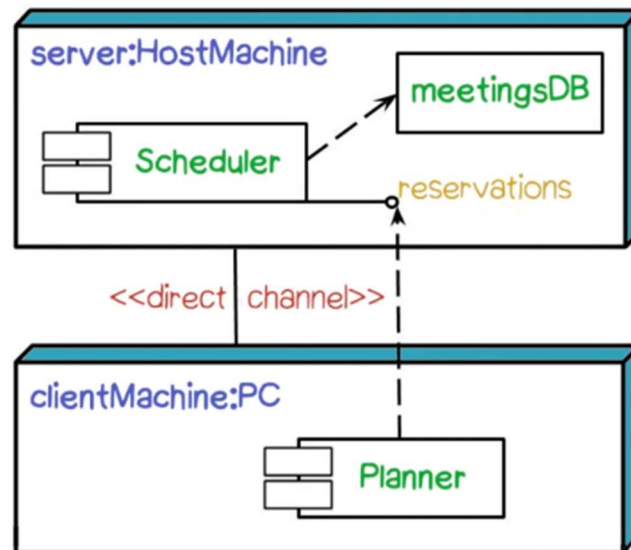
Example Component Diagram



The above component diagram shows classes or packages as the components, and how they interrelate to other components via interfaces. This type of diagram shows the relationships between components at a higher-level view of the system than a single class or component diagram.

Deployment Diagram – According to the UML Reference Manual, “Configuration of run-time processing nodes and the component instances and objects that live on them,” meaning that the deployment of parts of a system (components) to different computers/servers (nodes) and their dataflow are diagrammed in this document. “Node” denotes a computational device, and “arcs” indicate communications.

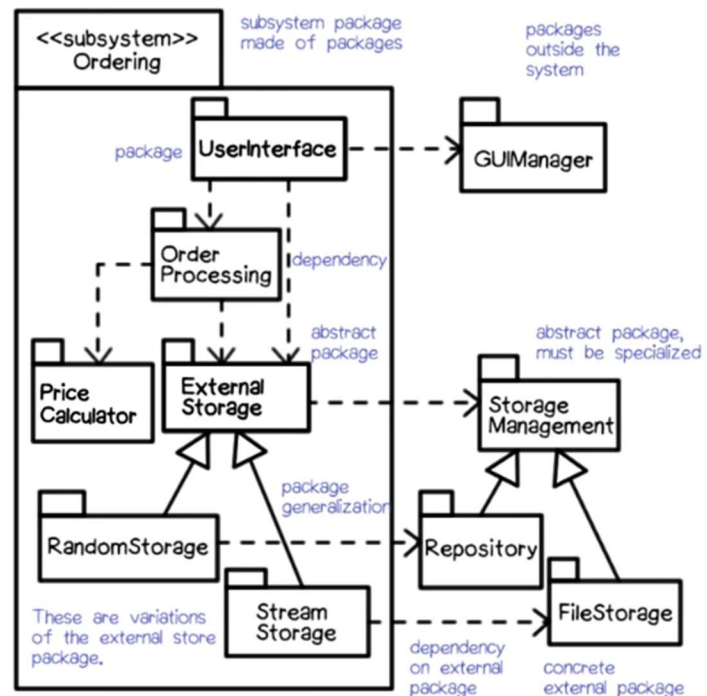
Example Deployment Diagram



The example deployment diagram above shows system components distributed across different machines (nodes) with information channels available to relay data across separate machines to different components hosted on them via “interfaces” in the code.

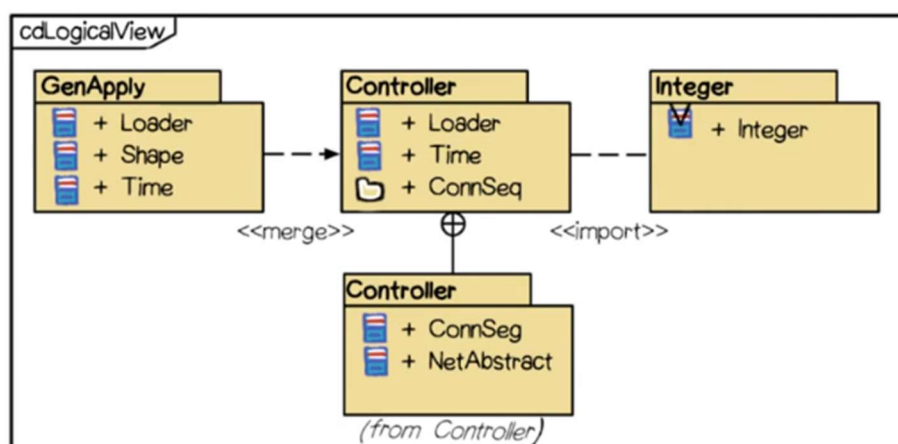
Packages – based on the Java packages organizing mechanism, packages are bundles of related code within the same “namespace” that tend to comprise a component with a singular function (separation of concerns). “System” is the top-level package, and dependency arrows between packages indicate the existence of dependencies between constituents.

Example Class Diagram with Packages (UML 1.5)



Traditional package diagram using UML 1.5, with multiple sub-packages contained within the “subsystem” package.

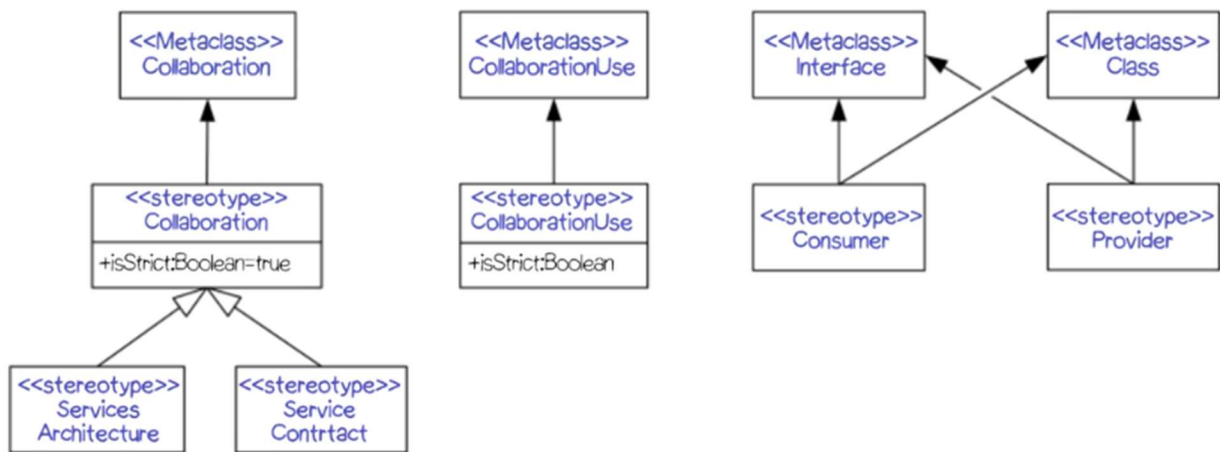
Example Package Diagram (SparxSystems)



This second example of a package diagram uses the UML 2.0 standard with software from SparxSystems. The same information is displayed, but consolidated to expandable lists within package symbols.

Profile Diagram – higher-level UML model of aggregated/connected UML class models that shows a top-level, comprehensive view of a system. This can be adjusted as necessary using custom symbols, relationships, etc.

Example Profile Diagram



The three UML profile diagrams provided above show top-level views of a system, and give a complete representation of the system when taken all together. The relationships can still show generalizations or associations, or even other non-standard relationships. There can be markings like `<<>>` that denote custom information as well.



Match up each of the UML Structure Diagram types with their descriptions:

d	Class	a. static structure at a particular time
f	Composite Structure	b. organization of physical software components
b	Component	c. logical groupings and dependencies
g	Deployment	d. components and structural properties
a	Object	e. extensions to the UML meta model
c	Package	f. internal structure and possible interactions
e	Profile	g. physical system resources and how they map to hardware

Subtitles/closed captions (c)

This quiz gives a good overview of the different types of UML Structure Diagrams along with their general definitions. There is a large amount of overlap in the structures covered by the diagrams, as outlined by the definitions.

UML structure diagrams that could be used to convey system architecture include Class Diagrams, Component Diagrams, Deployment Diagrams, and Package Diagrams. Internal and detailed implementation information is not relevant to higher-level system architecture diagrams.

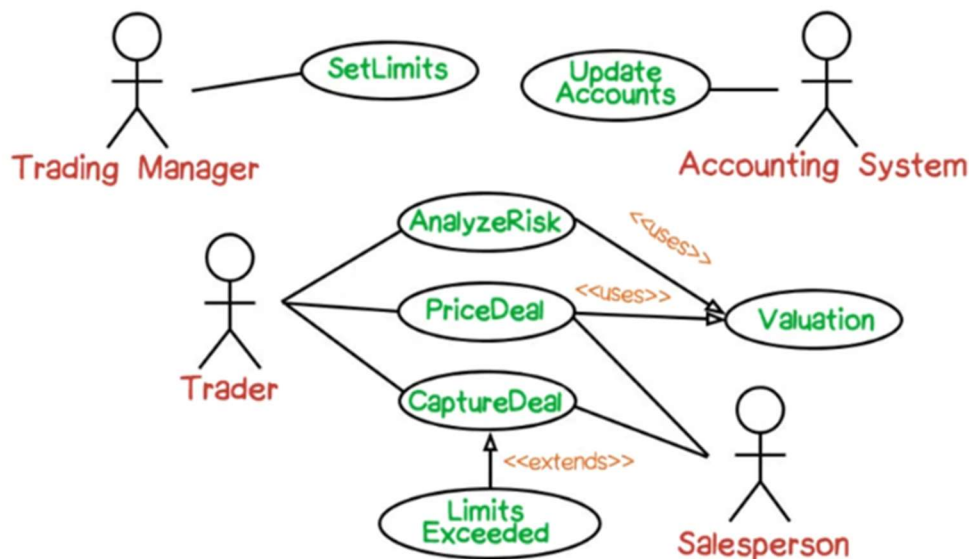
Behavior Diagrams – the second main category of UML diagrams, after structural diagrams. Behavior diagrams are focused on specific uses (behaviors/use cases) of the system, with each diagram only governing a specific usage or behavior. As a result, multiple behavior diagrams of the same type show all system usage scenarios when taken together, but not individually.

Use Case Diagram – diagram that shows a single “use case,” which is a sequence of user-visible actions along with system responses. It’s a story of how a system deals with a particular user interaction. This diagram should be extended to include the set of all use cases, adding more as they are generated or required.

Use cases are useful for eliciting system requirements and organizing development activities. The use case diagrams depict the relationships among system actors and individual use cases.

In the use case diagram, stick figures denote individual actors, mainly the user but potentially external actors like other systems or devices. Ovals denote individual use cases. Lines between actors and ovals without annotations indicate participation but not specific relationships/actions. The main relationships/actions governed by annotated lines are <<extends>> and <<uses>>, with the first giving specific edge cases for the original use case and the second using the other use case like a subroutine.

Example Use Case Diagram



The example use case diagram above shows the basic use-cases for a system involving different users, as well as an external system (accounting system), that are depicted by stick figures. The actual behaviors/uses are captured within ovals, and the standard <<extends>> or <<uses>> relationships are marked where they occur.

Use cases may be conveyed with unstructured text or in a table, demonstrated below with two images describing the same use case.

Example Use Case

Foster wants to buy a book. He goes to the Amazon web site and browses to find his book. He adds it to his shopping cart. He then goes to the check-out page where he provides his billing information and confirms his purchase.

Above is an unstructured text description of a single use case for a system.

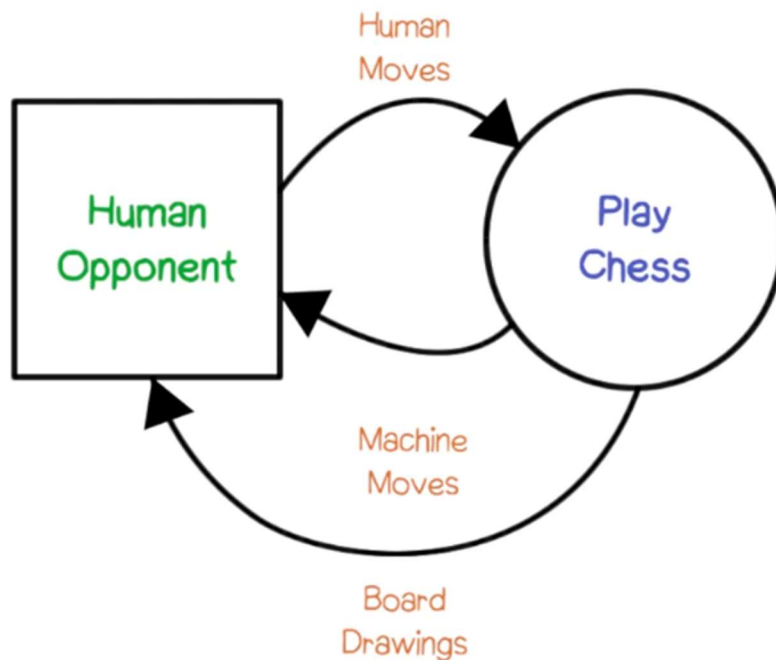
Tabular Version of Example

Actor	Action	Object
Foster	browses	Amazon website
Foster	selects	book
Foster	adds	book, shopping cart
Foster	checks out	
Amazon	requests	billing information
Foster	provides	name, address, credit
Amazon	verifies	credit information
Foster	confirms	purchase

Above is a structured, tabular description of a single use case for a system, identical to the previous use case outlined by the text description.

Context diagram – A non-UML diagram that gives an overview of the dataflow into/out of a system (across the system border) and shows the specific dataflows, processes, and actors surrounding the system.

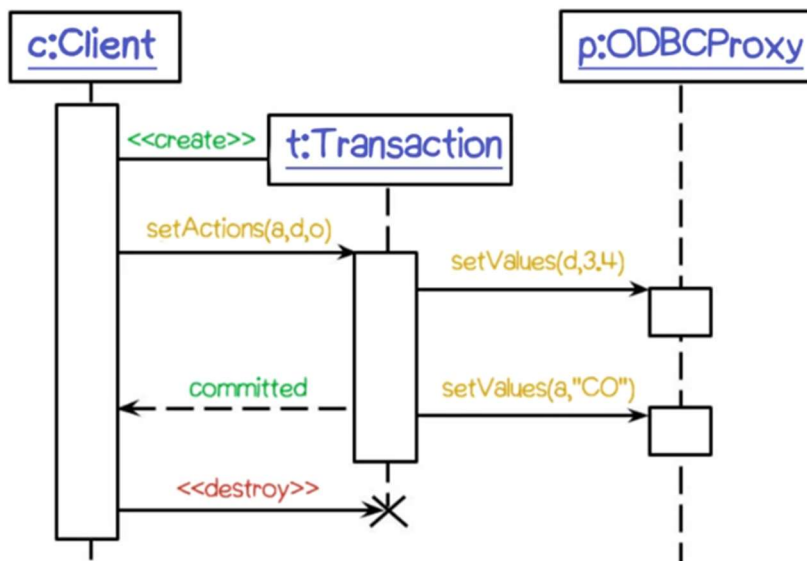
Example Context Diagram



The above context diagram shows how a chess-playing system interacts with the external environment – namely which data flows into and out of the system, and who the external actors are.

Sequence Diagram – one of the most popular UML behavior diagrams, it conveys a single use case and shows the different objects/services within the system and messages passed between them to complete the functions for the given use case.

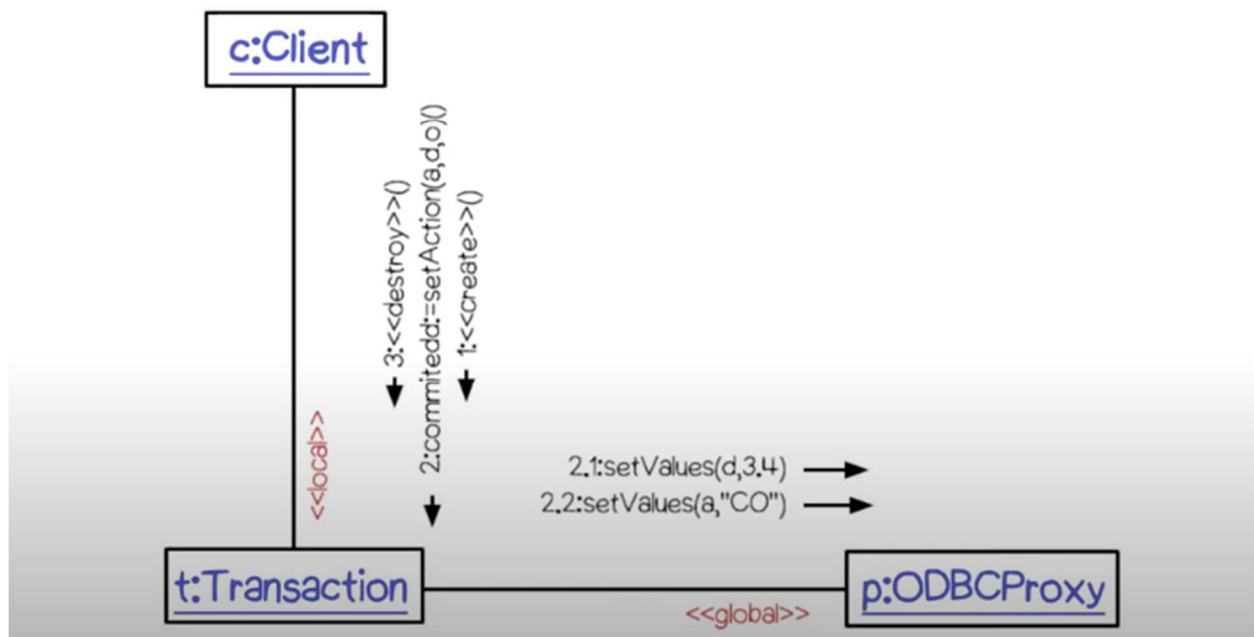
Example Sequence Diagram



This sequence diagram shows the columns that denote objects/services within the system, and the horizontal lines give the messages transferred between these objects/services in order. The temporal ordering is top-down, left to right for these messages, and is taken from traditional telephony diagrams. The semantics of this diagram are equivalent to the communication diagram (coming up next), so they show the same content but in a different form.

Communication diagram – UML object diagram (behavior diagram) showing ordered interactions between objects/services instead of links, showing the same content as a sequence diagram.

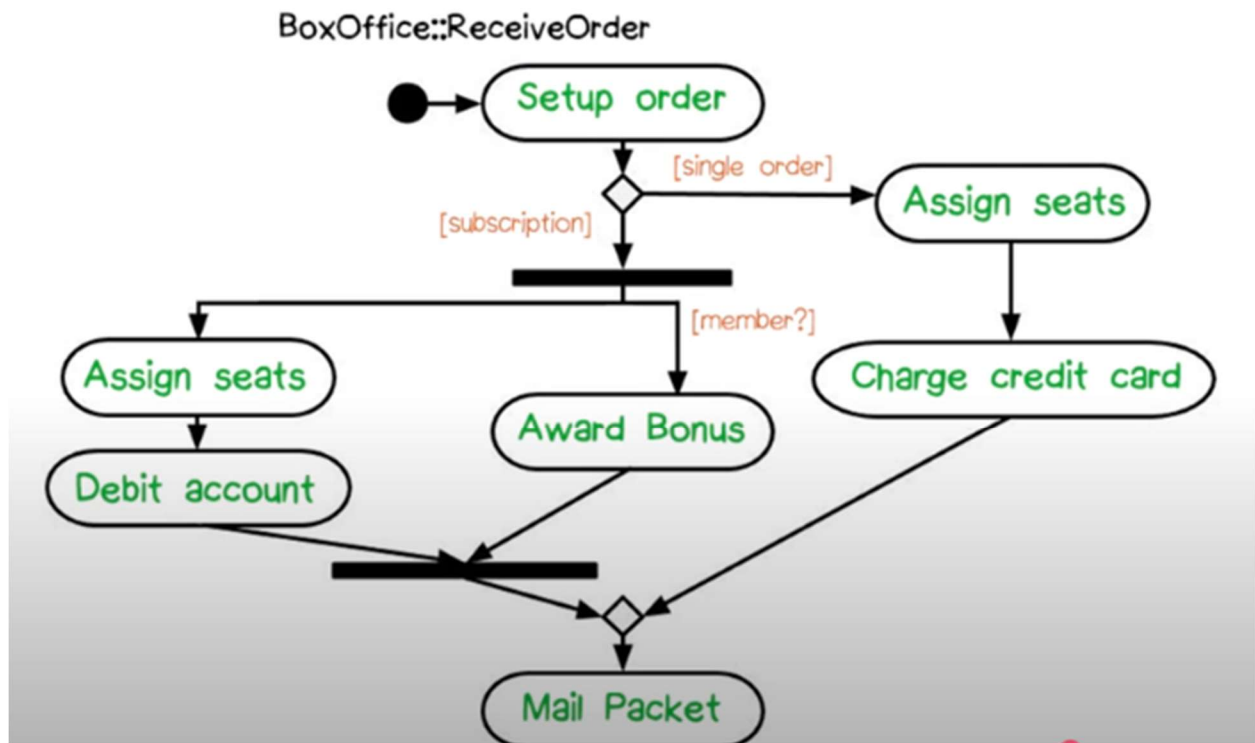
Example Communications Diagram



This communication diagram above gives the same information about the system for the same use case as the previous sequence diagram, but in a different format. The messages are represented as ordered interactions between objects/services, and numbered in the correct order of execution.

Activity diagram – A UML behavior diagram specifically designed to deal with concurrency and synchronization (which are not handled by the sequence or communication diagrams well), an activity diagram represents the different states of the system during potentially simultaneous actions, originally based on Petri nets. The state transitions only on the completion of an action/event, serving as a variant of a state diagram.

Example Activity Diagram

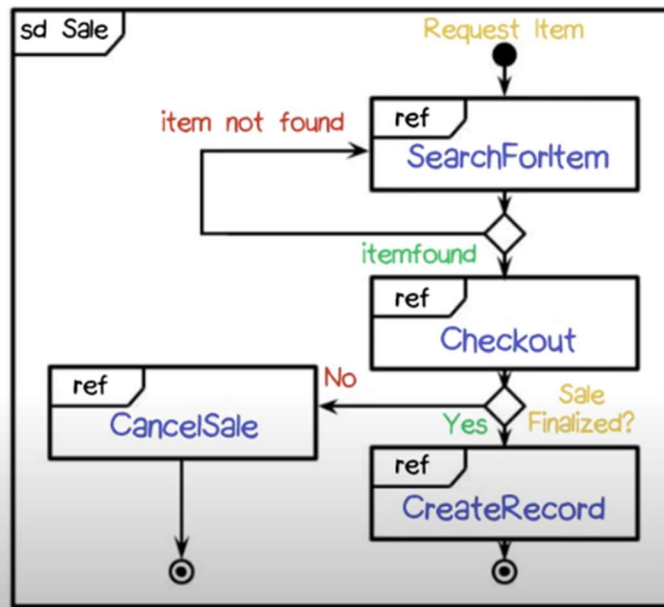


This activity diagram shows diamonds for splits/joins in activity paths, and also solid black rectangular bars for the start/end of concurrent operations (parallel activities). The black “token” at the top represents the start point, and moves through the diagram from the top down.

Two less-frequently-used behavior diagrams in UML include the Interaction Overview Diagram and the Timing Diagram.

Interaction Overview Diagram – higher-level diagram where the nodes in the diagram correspond to other behavior diagrams (such as sequence, communication, etc.).

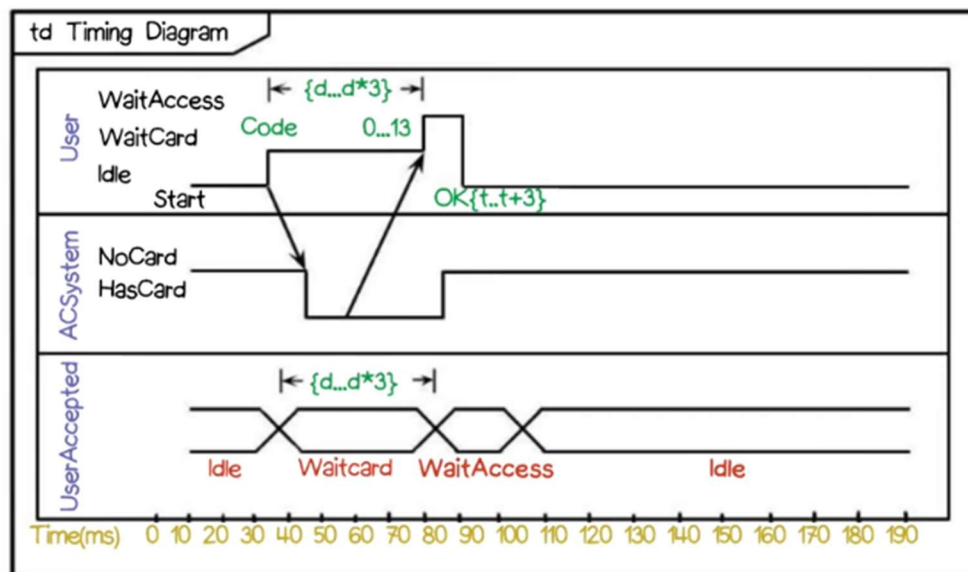
Interaction Overview Diagram



An example of an interaction overview diagram, “ref” denotes a specific interaction occurrence. Those sub-interactions could reference any UML behavior diagram (sequence, communication, timing, etc.)

Timing Diagram – another uncommon UML behavior diagram, timing diagrams show exact sequences of events and their duration for a given system behavior (use case). The diagrams are read from left to right, and are similar to timing diagrams used for microchips to time signals across the chip.

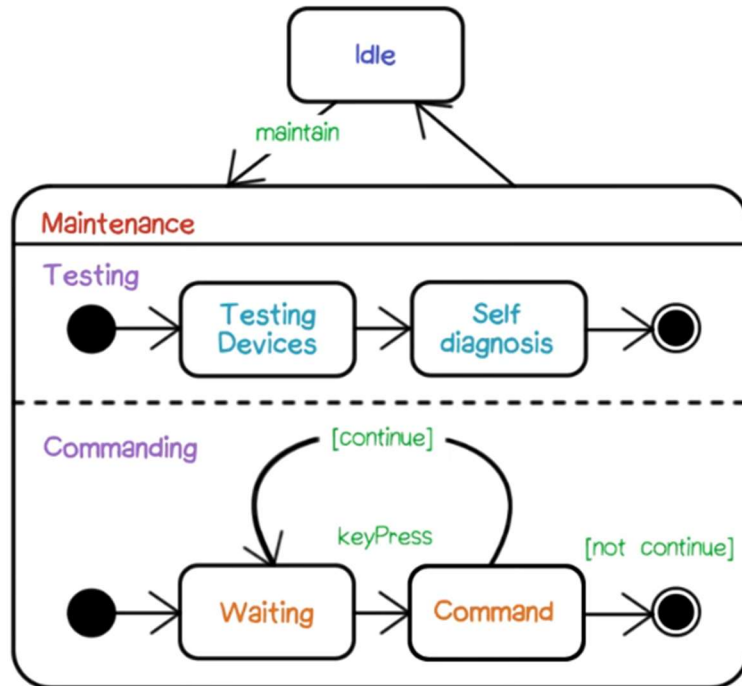
Timing Diagram



The above timing diagram shows events executing in order and duration across the system and its components during the fulfillment of a single use case.

State Diagrams – Most powerful and most complex type of UML behavior diagram (also called StateCharts). They show extended finite state machines that can depict aggregation, concurrency, history, broadcasting events, etc. They will be covered in more detail in a complete lesson later, with an overview given here.

Example State Machine Diagram



Two states are shown in the state machine diagram above, idle and maintenance, with two more sub-states illustrated within the maintenance state. Events flow from the black dot start point on the left to the black dot end point on the right, following the arrows. A back loop between events is shown in the commanding sub-state that can continue indefinitely or on a condition like a while loop in code.

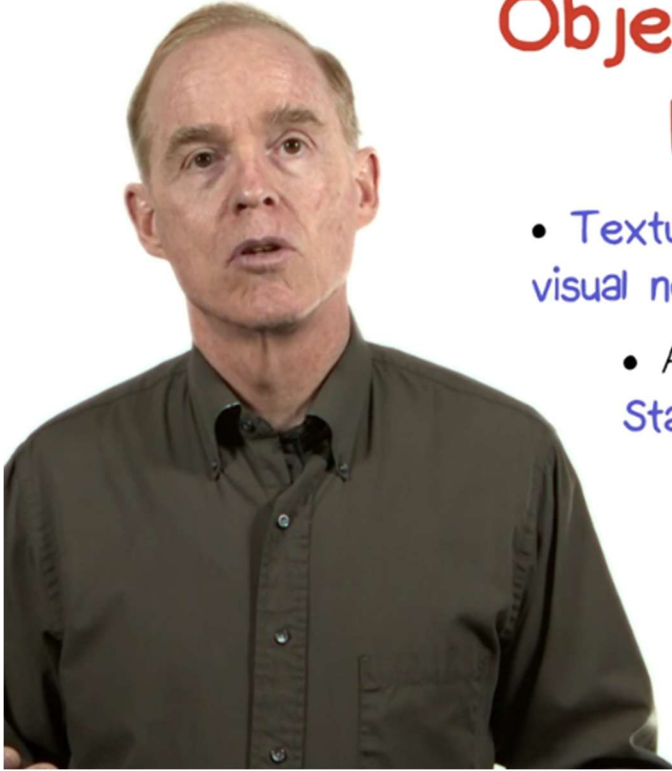


Match the following **behavior diagrams** with their descriptions:

3	Activity	1. system functionality provided to external actors
5	Sequence	2. dynamic behavior in response to stimuli
6	Communication	3. flow of control from activity to activity
4	Interaction Overview	4. synthesis of lower-level Activity Diagrams
7	Timing	5. interaction of classes of message exchange
1	Use Case	6. object interaction of numbered messages
2	State	7. rotated sequence diagram

The different types of UML behavior diagrams are listed here, along with their general definitions.

Object Constraint Language (OCL) – an extension to UML that enable text-based descriptions and constraints to be added to the standard diagrams.



Object Constraint Language

- Textual extension to UML's visual notation
- Applicable to Class and Statechart diagrams
- First-order predicate logic + diagram navigation + collection classes
 - Enables more precise specifications

The main purpose of the OCL extension to UML diagrams is precision; to enable more precise specifications for elements of the diagrams than available with standard UML.

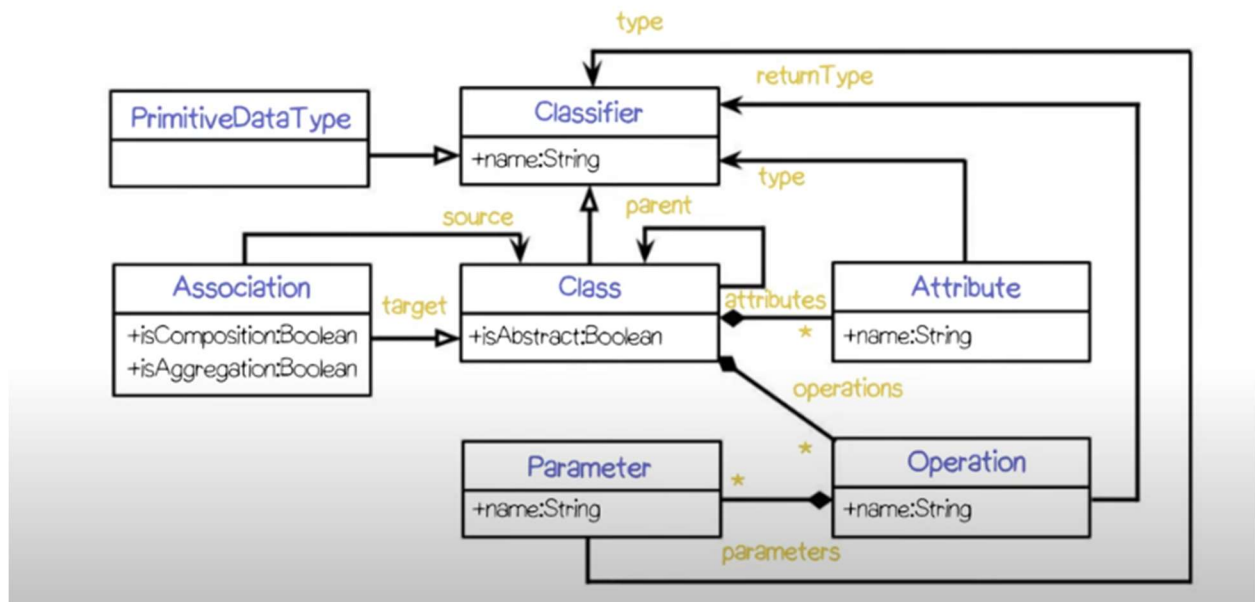
Example OCL

```
context Account::deposit(Real : amount)
pre: amount > 0
post: balance = balance@pre + amount
```

The above example of OCL text-based conditions for a class gives the class as "Account," the operation as "deposit," the precondition (which must be satisfied before the operation commences) that "amount" is positive and greater than zero, and the postcondition that the balance after the operation concludes is the balance before the operation starts added with the "amount" provided by the operation.

UML Metamodel – a definition of the UML language using UML constructs and diagrams, forming a “meta” model of UML. This metamodel can be extended using custom features and annotations – called a “profile.”

Class Model of UML Metamodel



UML MetaModel described using a UML Class Diagram to include the basic components of the UML language (Classes, Attributes, Operations, etc.)

You can't design a complex system without having some idea of what it's supposed to do – what problem it is trying to solve.

Diagrams can help you express that understanding and express your solution to that problem.

The more precisely you understand a problem, the fewer subsequent problems you will have with that system's history.